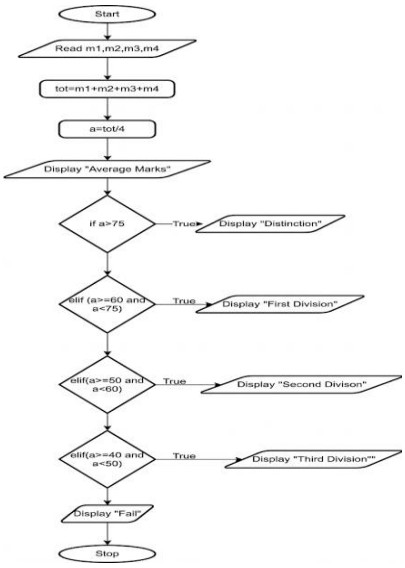
## Problem Statement:-

Write a program to calculate the total marks, aggregate percentage, and grade of a student based on marks in four subjects.

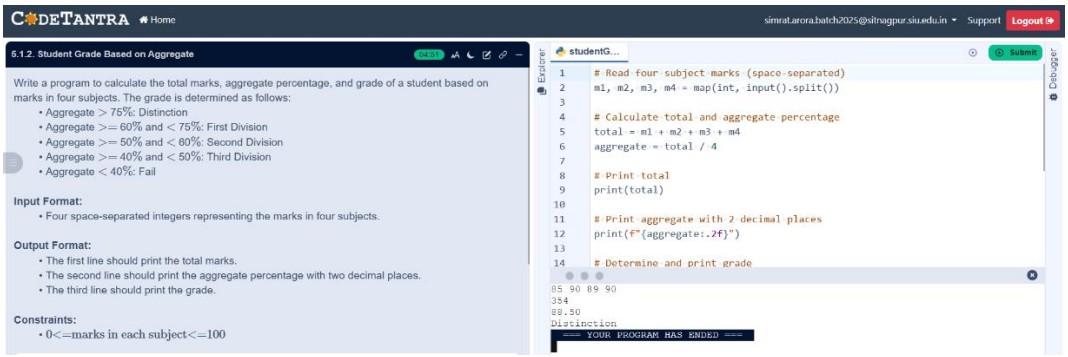
## Algorithm:-

1. Start
2. Read marks m1, m2, m3, m4
3. total = m1 + m2 + m3 + m4
4. percentage = total / 4
5. If percentage > 75 → Print **"Distinction"**
6. Else if percentage ≥ 60 → Print **"First Division"**
7. Else if percentage ≥ 50 → Print **"Second Division"**
8. Else if percentage ≥ 40 → Print **"Third Division"**
9. Else → Print **"Fail"**
10. Print total and percentage
11. Stop.

## Flowchart:-




## Execution:-



# Experiment 06:-

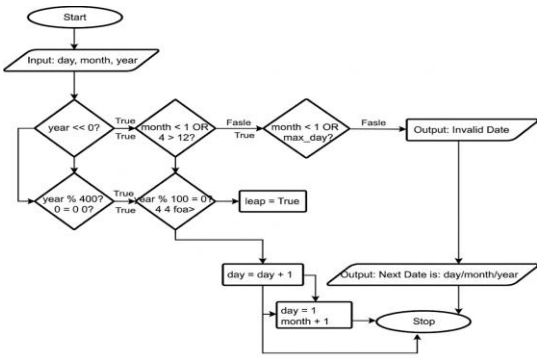
## Problem Statement:-

Write a Python program to check if a given date is valid. If the date is valid, print the next day's date (incremented date). If the date is invalid, print "Invalid Date".

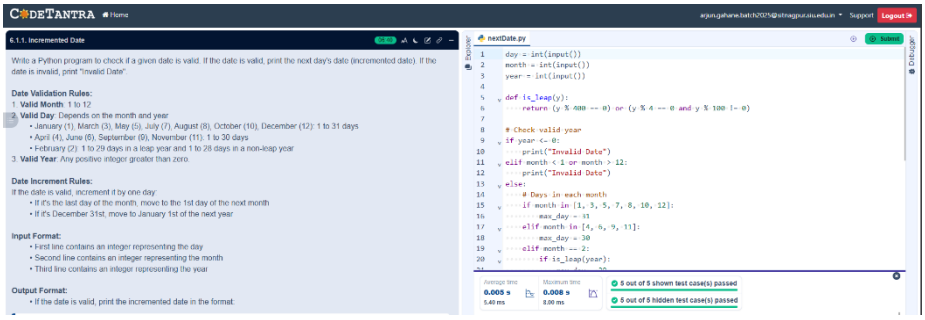
## Algorithm:-

1. Start.
2. Read d, m, y
3. If invalid y or m → Print **Invalid**, Stop
4. Find leap year
5. Set max_day for month (Feb = 28/29)
6. If invalid d → Print **Invalid**, Stop
7. d++
8. If d > max_day → d=1, m++
9. If m > 12 → m=1, y++
10. Print **Next Date**
11. **Stop.**

## Flowchart:-




### Execution:-



## Problem Statement:-

Write a Python program to calculate the factorial of a number n using loops.

## Algorithm:-

1. Start
2. Read a number n from the user.
3. If n < 0

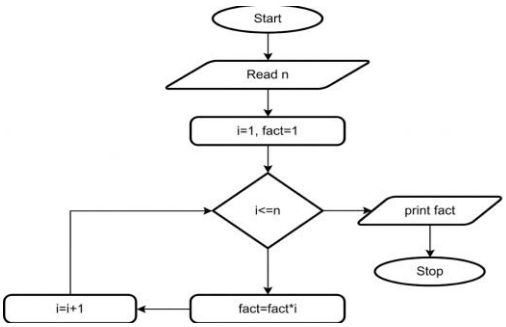
   → Print "Factorial not defined for negative numbers"

   → Stop
4. Set fact = 1
5. Set i = 1
6. Repeat while i ≤ n

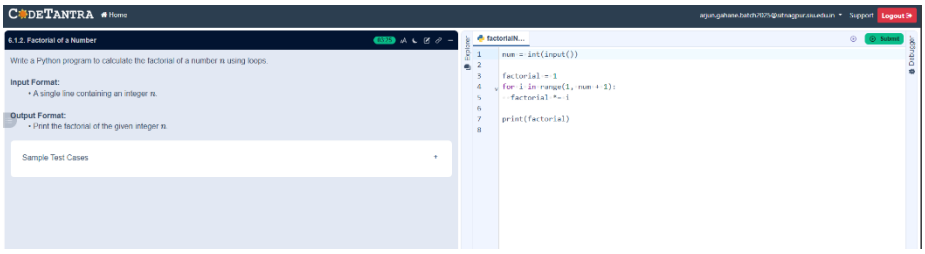
   → fact = fact × i

   → i = i + 1
7. Print "Factorial of n is fact"
8. Stop.

## Flowchart:-




**Execution:-**



# Experiment 07:-

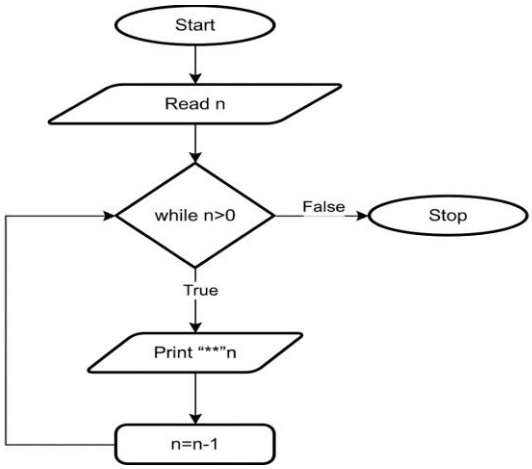
## Problem Statement:-

Write a Python program to print an inverted right-angled triangle star pattern.

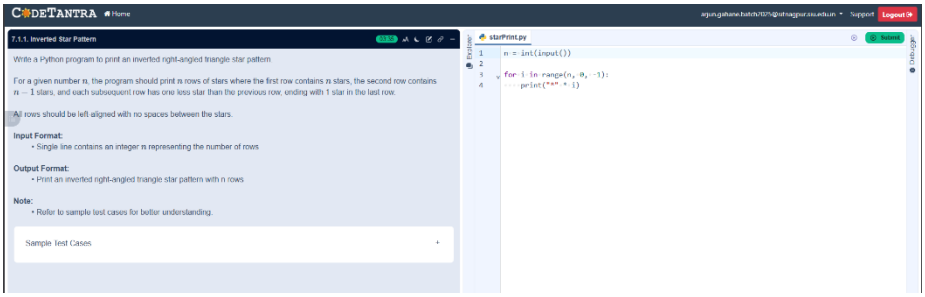
## Algorithm:-

1. Start.
2. Input value n
3. While n > 0:
4. Print "*" repeated n times
5. Decrease n by 1
6. Stop

## Flowchart:-




### Execution:-



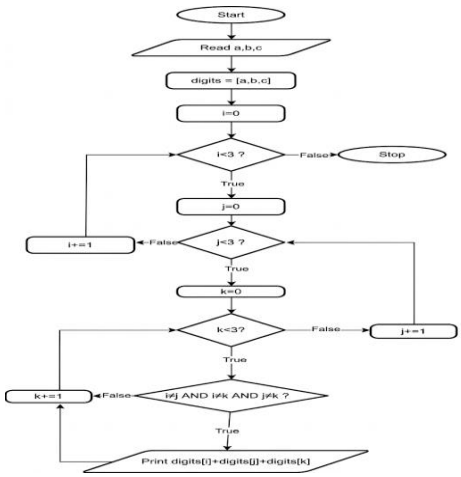
## Problem Statement:-

Write a Python program to accept three digits and print all possible 3-digit combinations that can be formed using those three digits. Each digit should be used exactly once in each combinations.

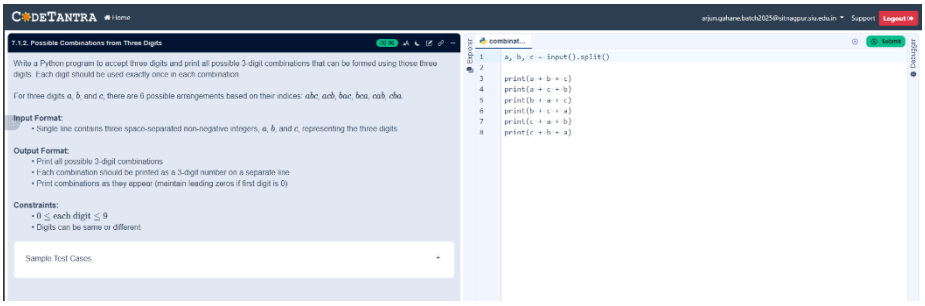
## Algorithm:-

1. Start
2. Input three values a, b, c.
3. Store them in a list
   digits = [a, b, c]
4. Initialize loop variable i from 0 to 2.
5. Inside that, initialize loop variable j from 0 to 2.
6. Inside that, initialize loop variable k from 0 to 2.
7. Check condition:
   if ($i \neq j$ AND $i \neq k$ AND $j \neq k$)
   If true → Print
   digits[i] + digits[j] + digits[k]
8. Repeat until all loops finish.
9. Stop.

## Flowchart:-




## Execution:-



# Experiment 08:-

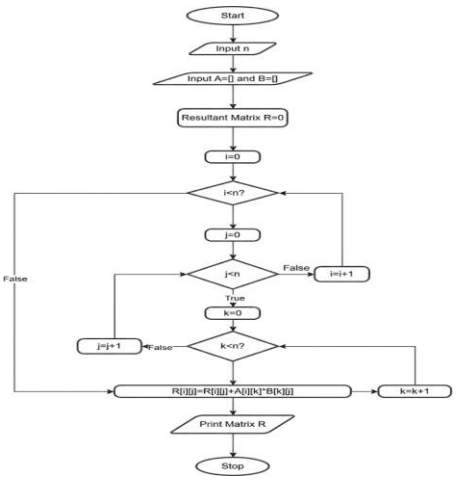
## Problem Statement:-

Write a Python program to perform the multiplication of two square matrices. The program should take two matrices as input and output their product.

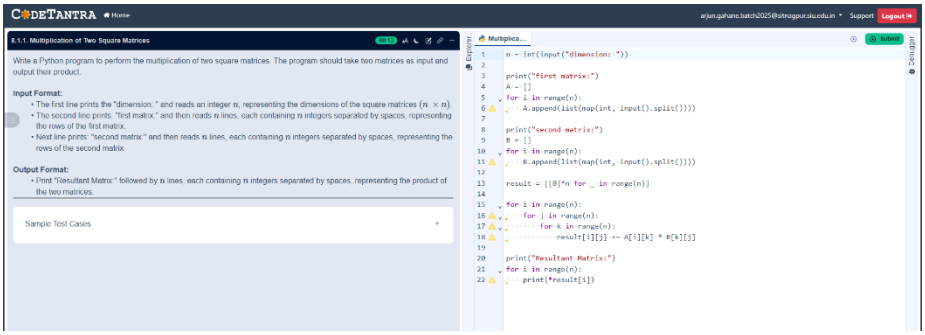
## Algorithm:-

1. Start.
2. Input the order n of the square matrices.
3. Input elements of Matrix A and Matrix B.
4. Create a result matrix R of size n × n and initialize all elements to 0.
5. Initialize loop variable i from 0 to n−1.
6. Inside that, initialize loop variable j from 0 to n−1.
7. Inside that, initialize loop variable k from 0 to n−1.
8. Perform multiplication and addition: If inside loops → R[i][j] = R[i][j] + A[i][k] × B[k][j]
9. Repeat until all loops finish.
10. Print the result matrix R.
11. Stop.

## Flowchart:-




## Execution:-



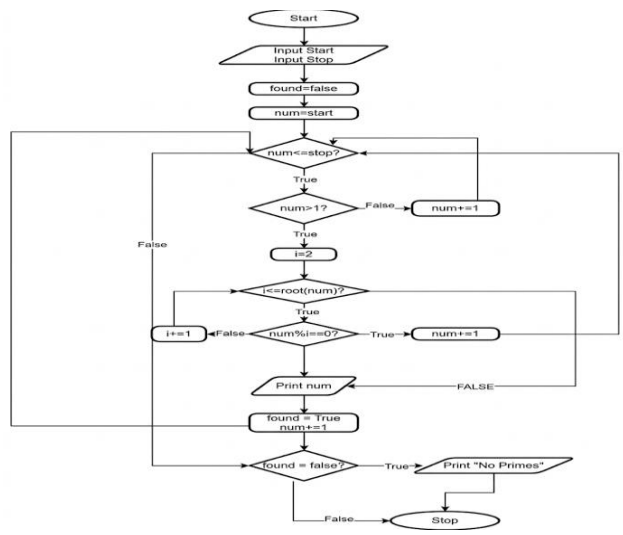
## Problem Statement:-

Write a Python program to print all prime numbers within a given range (inclusive of both start and end values).

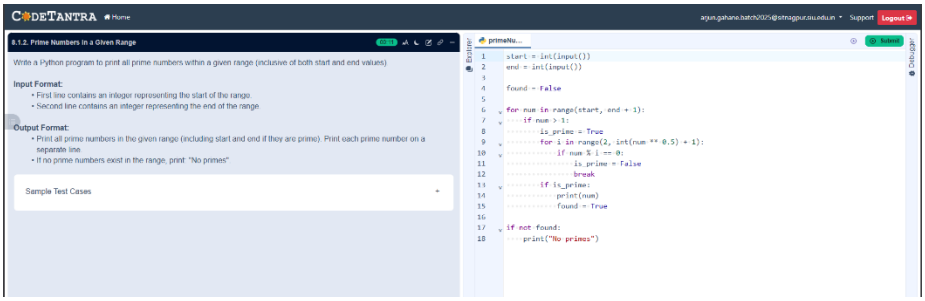
## Algorithm:-

1. Start
2. Input the values start and stop.
3. Set found = False.
4. Repeat for each number num from start to stop.
5. Check if num > 1.
6. Repeat for i from 2 to √num.
7. Check condition:
   If num % i == 0 → break (number is not prime).
8. If no divisor is found, then:
   Print num
   Set found = True.
9. After the loop, check found.
10. If found == False → Print "No primes".
11. Stop

## Flowchart:-




## Execution:-



# Experiment 09:-

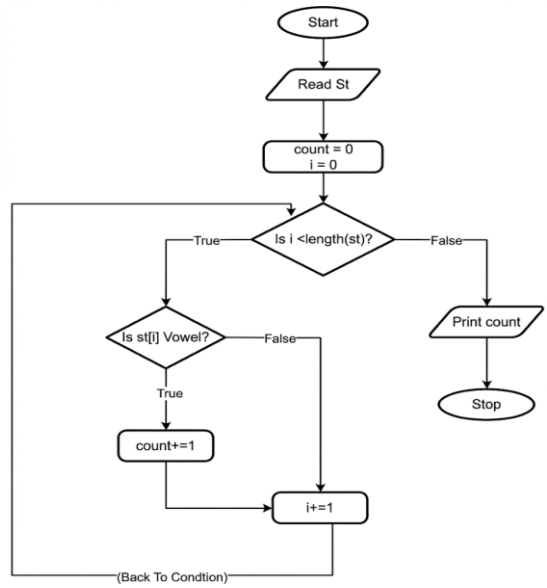
## Problem Statement:-

Write a program that takes a string as input and prints the total count of vowels in that string. Consider both uppercase and lowercase.

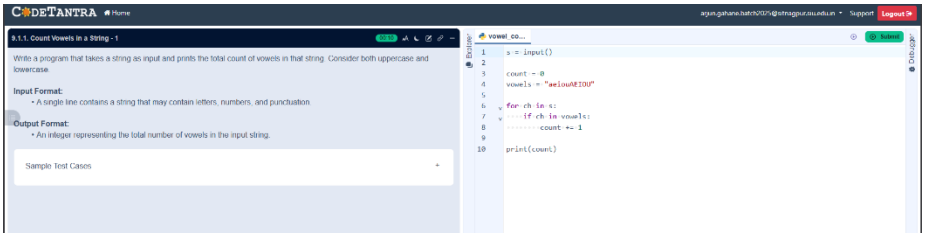
## Algorithm:-

1. Start.
2. Read string st
3. Set count = 0
4. Traverse each character of string
5. If character is vowel → count = count + 1
6. Print count
7. Stop

## Flowchart:-




## Execution:-



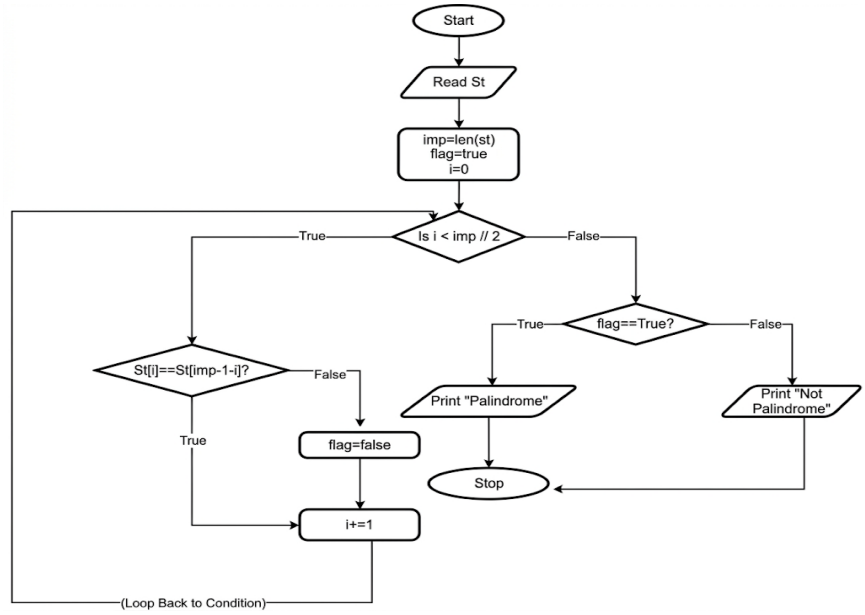
## Problem Statement:-

Write a Python program to take a string as input and check whether the given string is a palindrome or not. A palindrome is a string that reads the same forwards and backward.

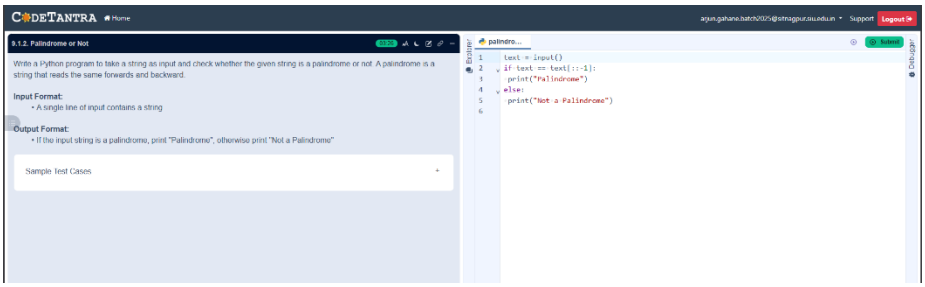
## Algorithm:-

1. Start
2. Read string st.
3. Find length n.
4. Set flag = True.
5. Compare characters from start and end..
6. If mismatch → flag = False
7. If flag true → Palindrome else Not Palindrome.
8. Stop.

## Flowchart:-




## Execution:-



# Experiment 10:-

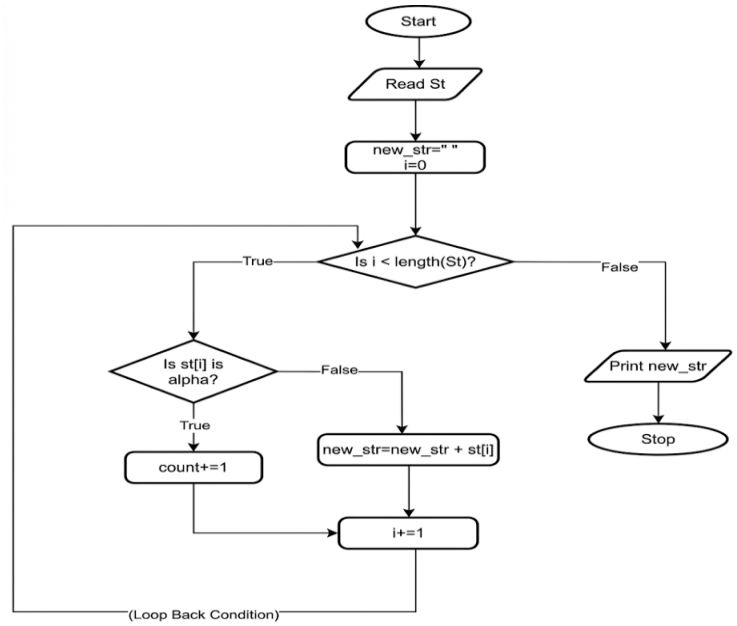
## Problem Statement:-

Write a Python program to remove all punctuation marks from a given string and print the resulting string.

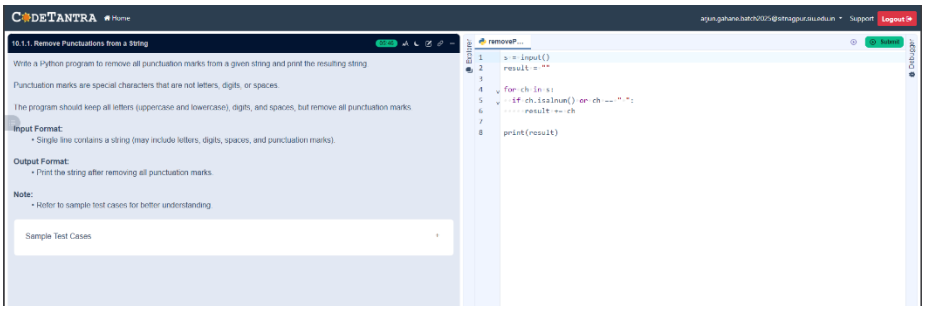
## Algorithm:-

1. Start..
2. Read string st.
3. Set new_str = "".
4. Traverse each character.
5. If character is alphanumeric or space → add to new_str.
6. Print new_str.
7. Stop.

## Flowchart:-




## Execution:-



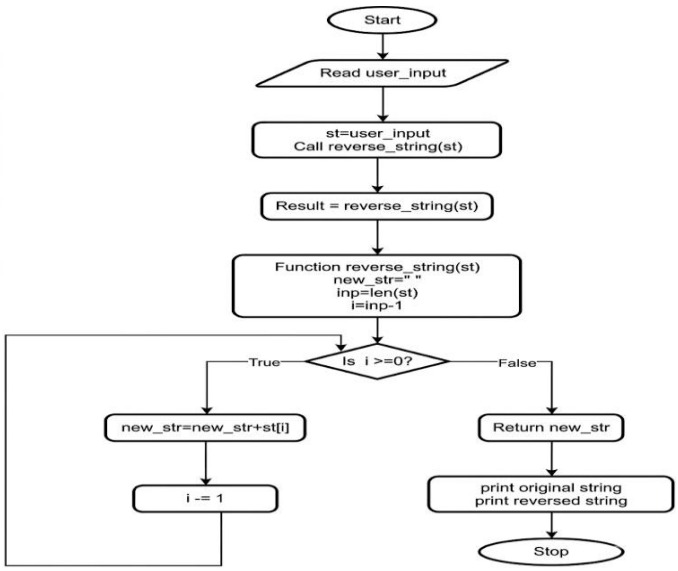
## Problem Statement:-

Write a Python program that prints the reverse of a given input string. The program uses a **reverse_string()** function that takes a string as input and returns its reversed string.

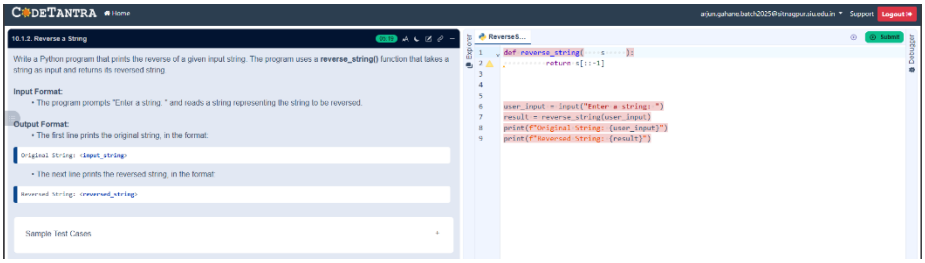
## Algorithm:-

1. Start
2. Define function to reverse string.
3. Initialize empty string.
4. Traverse string in reverse order.
5. Append characters to new string..
6. Return reversed string
7. Input string.
8. Call function.
9. Print original and reversed string.
10. Stop.

## Flowchart:-




## Execution:-



# Experiment 11:-

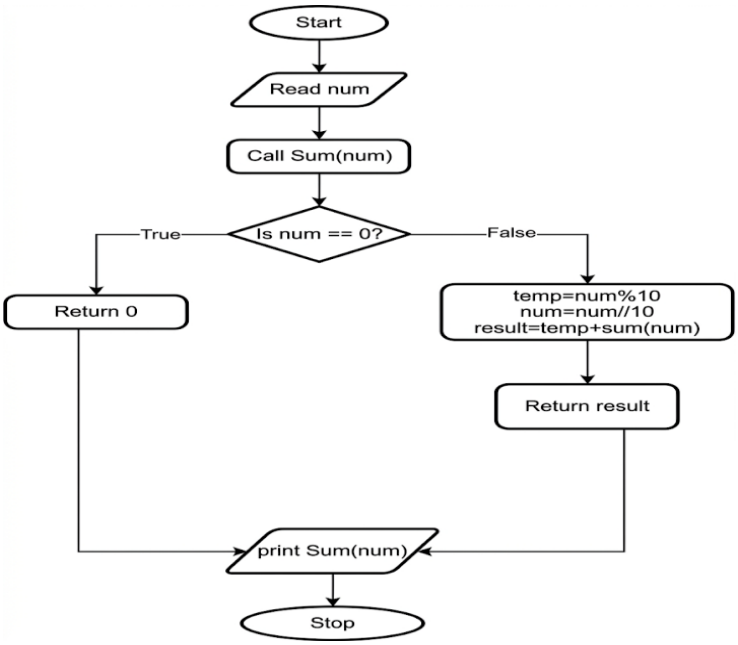
## Problem Statement:-

Write a Python program that reads an integer, calculates the sum of its digits, and prints the result.

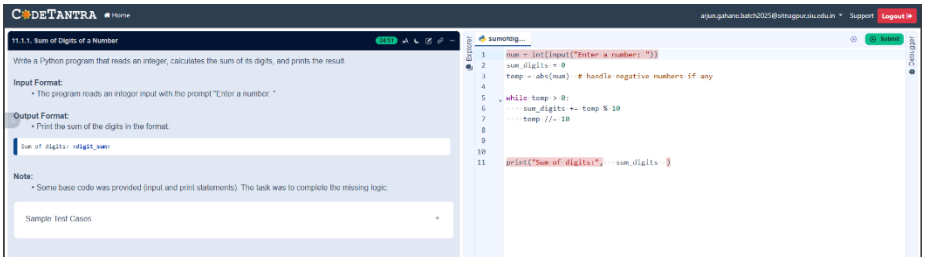
## Algorithm:-

1. Start.
2. Read number num.
3. If num == 0 → return 0.
4. Else → return last digit + sum of remaining digits..
5. Call function
6. Print result.
7. Stop.

## Flowchart:-




## Execution:-



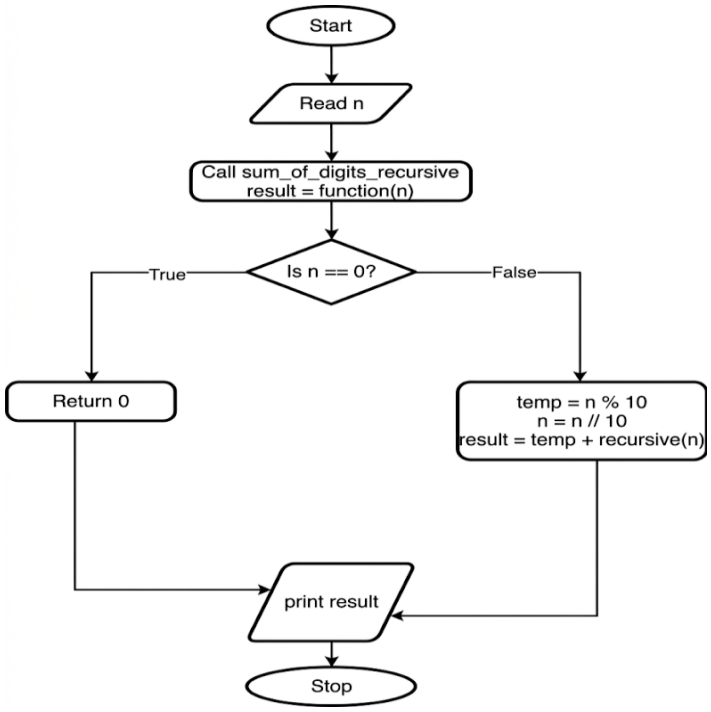
## Problem Statement:-

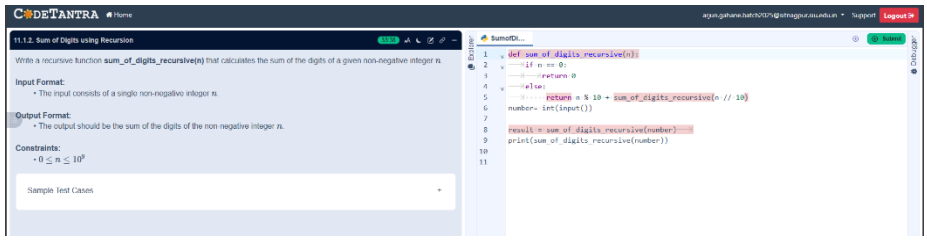
Write a recursive function **sum_of_digits_recursive(n)** that calculates the sum of the digits of a given non-negative integer .

## Algorithm:-

1. Start
2. Read number n.
3. If n == 0 → return 0.
4. Else → return last digit + recursive call.
5. Call function.
6. Print result.
7. Stop.

## Flowchart:-




## Execution:-